Maze Game – Technical Documentation

Maze Game is a Python game built using Pygame. The player navigates through a procedurally generated maze while being chased by an AI-controlled monster that uses the A* pathfinding algorithm. The game features fog of war, smooth camera movement, configurable settings, background music, and a fullscreen UI.

Developers

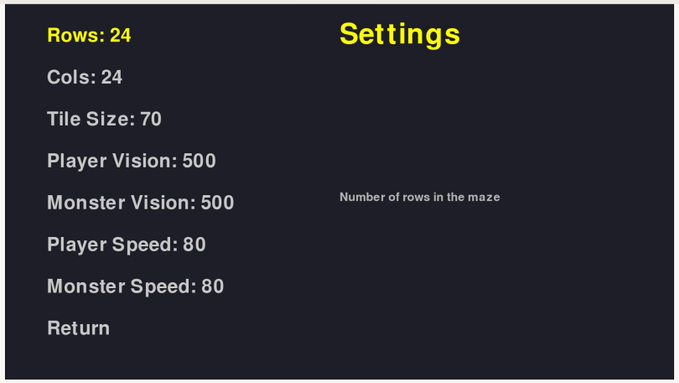
- George Assem Aziz 2301325
- Marawn Montaser Fakhry 2301343
- Ziad Ashraf Saban 2301320
- Ziad Ashraf Saad 2301279
- Amir Abdelrahim Ahmed 2300789

Overview

The project is divided into three main components:

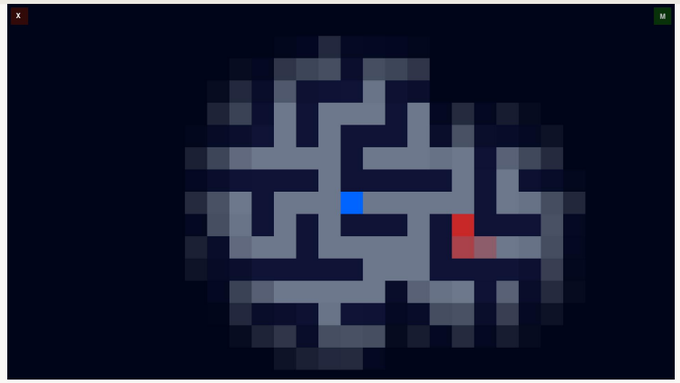
menu.py

Game configuration menu



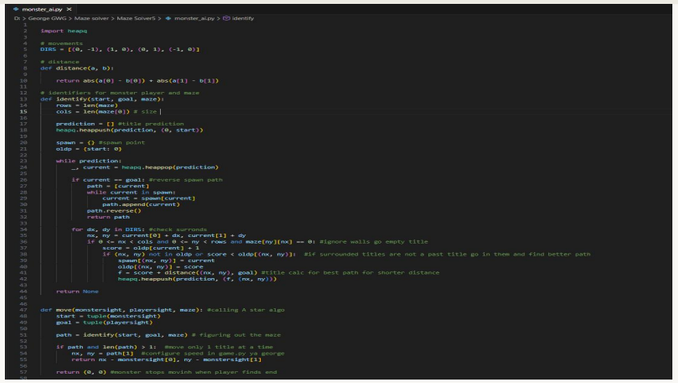
game.py

Core gameplay loop and rendering



monster_ai.py

Monster pathfinding and AI logic



Game Features

Procedural Maze Generation

With loops for increased complexity.

A* Pathfinding AI

Monster chases the player using an efficient algorithm.

Fog of War

Smooth vision transitions for an immersive experience.

Smooth Camera Movement

Camera is always centered on the player.

Fullscreen, Borderless Window

Optimized for an uninterrupted gaming experience.

Configurable Settings

Adjust maze size, tile size, and vision radius.

Background Music

With a convenient mute toggle.

Tranquil
by. Jeorje

UI Buttons

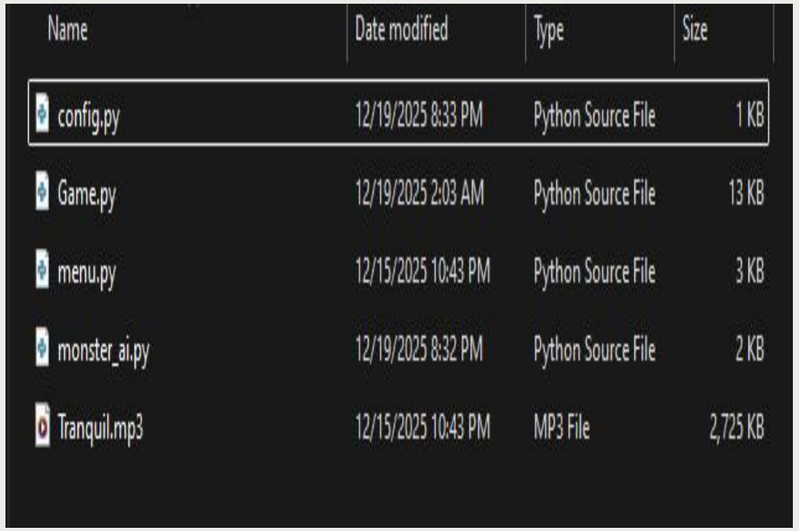
Exit and mute buttons for easy control.

X

M

File Structure

```
maze_game/
│
├── menu.py # Settings menu
├── game.py # Main game logic
├── monster_ai.py # AI pathfinding logic
├── config.py # Auto-generated configuration file
├── Tranquil.mp3 # Background music
```



Name	Date modified	Type	Size
config.py	12/19/2025 8:33 PM	Python Source File	1 KB
Game.py	12/19/2025 2:03 AM	Python Source File	13 KB
menu.py	12/15/2025 10:43 PM	Python Source File	3 KB
monster_ai.py	12/19/2025 8:32 PM	Python Source File	2 KB
Tranquil.mp3	12/15/2025 10:43 PM	MP3 File	2,725 KB

menu.py – Settings Menu

The menu system allows the player to configure game parameters before starting the game.

Purpose

- Adjust maze size and gameplay parameters
- Save settings to config.py
- Launch the main game

Configurable Settings

Setting	Description
Rows	Number of maze rows
Cols	Number of maze columns
Vision	Player visibility radius (tiles)
Tile Size	Size of each maze tile in pixels

Controls

- Up / Down Arrow – Navigate settings
- Left / Right Arrow – Modify values
- Enter – Save configuration and start game
- Esc – Exit menu

Configuration Saving

The `save_config()` function writes the selected values to `config.py`, which is imported by `game.py` at runtime.

game.py – Core Gameplay

This file contains the main game loop, rendering logic, player movement, and interactions.

01

Initialization

Initializes Pygame and audio mixer. Loads background music and starts looping playback. Loads configuration values from `config.py`.

02

Maze Generation

The maze is generated using a depth-first search (DFS) algorithm with bias against long straight paths and additional random loops for complexity. Functions involved: `generate_maze()` – Creates the main maze structure, `add_loops()` – Adds alternative paths.

03

Start and Exit Points

Start position is fixed at (0, 0). Exit is determined using Breadth-First Search (BFS) to find the farthest reachable tile.

04

Player Movement

Controlled using WASD or Arrow Keys. Movement is tile-based and time-delayed (`MOVE_DELAY`). Player position is tracked along with two ghost positions for visual trailing.

05

Monster Behavior

Monster starts at the maze exit. Moves toward the player using A* pathfinding. Speed is controlled by `monsterspeed`. Leaves a short visual trail.

06

Camera System

Smooth camera interpolation toward the player. Camera movement is clamped to maze boundaries. Fog camera lags slightly for cinematic effect.

07

Fog of War

Darkness overlay covers unexplored areas. Circular vision radius around player. Smaller vision radius around monster. Smooth fade-out at edges.

08

UI Elements

Mute Button (Top-Right) – Toggles background music. Exit Button (Top-Left) – Closes the game.

09

Win and Lose Conditions

Win: Player reaches the exit tile. Lose: Monster reaches the player.

monster_ai.py – AI & Pathfinding

This module implements the monster's decision-making logic.

Algorithm Used: A*

The monster uses the A* search algorithm, which combines:

- Actual path cost from the start (g)
- Heuristic distance to the goal (h)

Heuristic Function

```
abs(x1 - x2) + abs(y1 - y2)
```

Manhattan distance is used since movement is grid-based.

Core Functions

`distance(a, b)` – Heuristic calculation

`identify(start, goal, maze)` – Computes optimal path

`move(monstersight, playersight, maze)` – Returns one-tile movement direction

The monster only moves one tile per update to allow speed control in `game.py`.

Controls Summary

Action	Key
Move	WASD / Arrow Keys
Mute Music	M
Exit Game	ESC or Exit Button

Future Improvements

→ **Multiple Monsters**

With different AI behaviors.

→ **Player Abilities**

Sprint, hide, traps, and more.

→ **Difficulty Scaling**

Adjust game difficulty dynamically.

→ **Mini-map or Memory-based Fog**

Enhanced navigation and exploration.

→ **Sound Effects and Animations**

For a richer sensory experience.

Dependencies and Conclusion

Dependencies

- Python 3.x
- Pygame

Install Pygame:

```
pip install pygame
```

- Ensure all project files are in the same directory.
- Run the menu file using: python menu.py
- Adjust settings if desired and press ENTER to start the game.

Testing Results

- **Maze Validity Test: Ensured all generated mazes are solvable.**
- **Monster Pathfinding Test: Verified that the monster successfully follows the player.**
- **Collision Test: Confirmed correct behavior when the player is caught or reaches the exit.**
- **Performance Test: Maintained stable performance at 60 FPS.**

All tests produced the expected results without critical errors.

Conclusion

This project demonstrates procedural content generation, real-time pathfinding AI, and advanced visual effects using Pygame. The modular structure allows easy expansion and experimentation with AI behaviors and gameplay mechanics.